PORTFOLIO SHOWCASE

# Food Ordering & Delivery Platform

**Full-Stack · Microservices · Event-Driven Architecture**

React (TypeScript) | Spring Boot | Kafka | PostgreSQL | MongoDB | Docker

Role: Full-Stack Engineer

## 01 PROJECT OVERVIEW

Dual-application food ordering ecosystem with microservice architecture

CraveRush is a comprehensive, production-ready food ordering and delivery ecosystem built entirely from scratch. It is composed of two distinct yet seamlessly integrated applications: a **Customer & Admin Platform** (CraveRush Core) and a **Delivery Partner Microservice**, communicating asynchronously over **Apache Kafka**.

| | CraveRush Core (App 1) | Delivery Microservice (App 2) |
|---|---|---|
| **Frontend** | React · TypeScript · Tailwind · Framer Motion · Zustand | React · TypeScript · Tailwind · React Context |
| **Backend** | Java · Spring Boot · Spring Security · Hibernate · Stripe | Java · Spring Boot · Apache Kafka Consumers |
| **Database** | PostgreSQL | PostgreSQL + MongoDB (Polyglot) |
| **Port** | :8080 | :9090 |

## 02 APPLICATION SCREENSHOTS

Real screens from the built platform

**Customer Interface — CraveRush Homepage**

Mobile-first dark-themed UI featuring hero banner with search, cuisine spotlights, recommended dishes, trending restaurants, exclusive offers, and community blog section.


*Fig. 1 — CraveRush Customer Homepage (Full Page View)*

**Platform System Architecture Diagram**

Component-level breakdown of the CraveRush frontend architecture — showing the hero section, stats block, cuisine filters, recommendation carousel, promotional blocks, and footer.

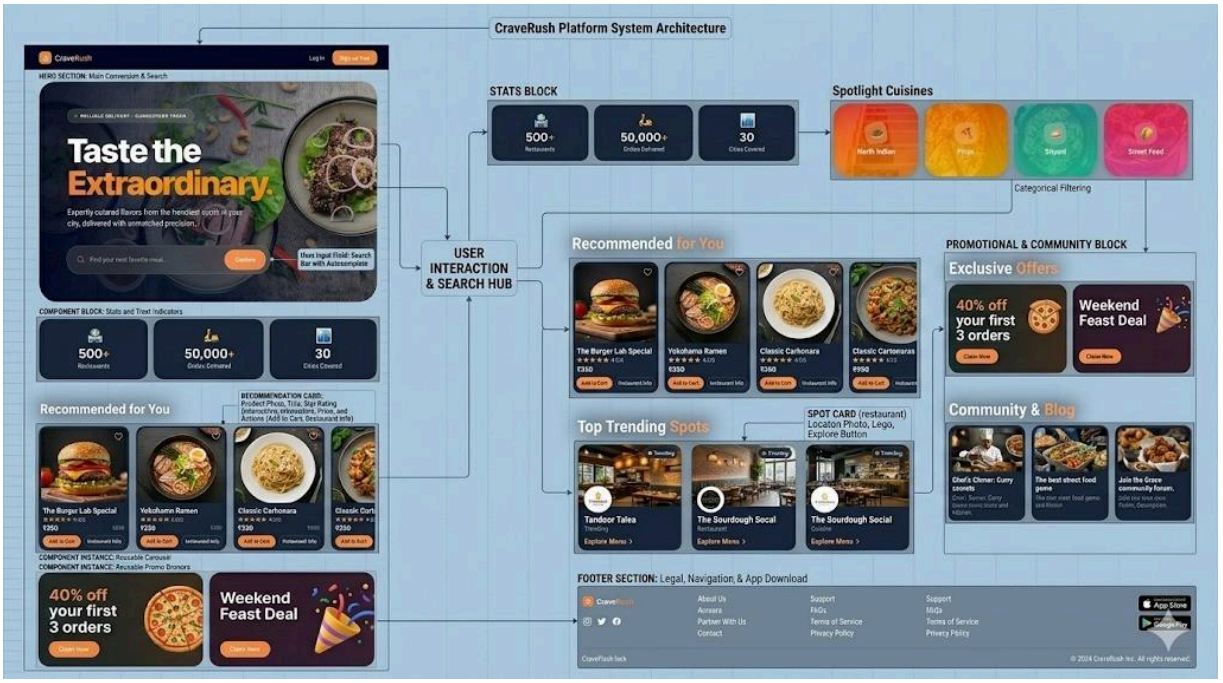


*Fig. 2 — CraveRush Frontend Component Architecture Map*

**Admin Hub — Analytics Dashboard**

Dedicated admin panel with real-time revenue metrics, order status distribution charts, monthly revenue trends, and a profit tracker with CSV export functionality.

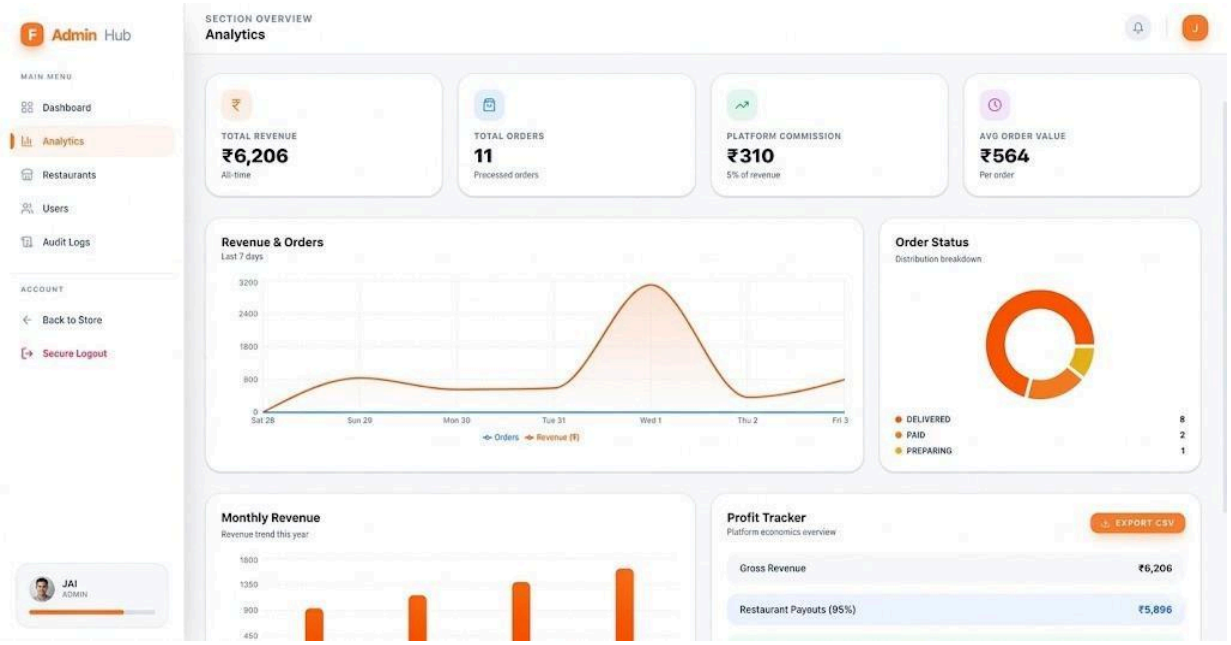


*Fig. 3 — Admin Hub: Analytics Dashboard (Revenue · Orders · Commission · Profit Tracker)*

## 03 CORE FEATURES

Both applications combined

| | |
|---|---|
| **Premium UI/UX**<br>Mobile-first dark theme, smooth Framer Motion animations, glassmorphism cards, and orange gradient accents delivering a Dribbble-grade aesthetic. | **Kafka Event Bus**<br>Core app publishes order.confirmed events; Delivery microservice consumes them across multiple partitions for high-throughput, resilient processing. |
| **JWT Authentication**<br>Stateless token auth via Spring Security filter chains; Axios interceptors silently attach Bearer tokens on every protected API request. | **Assignment Engine**<br>Matches confirmed orders to available drivers based on real-time state: Offline / Available / En-Route — with proximity-aware allocation logic. |

| | |
|---|---|
| **Stripe Payments**<br>Seamless PCI-compliant checkout integrated via the Stripe API with real-time payment success/failure feedback during order placement. | **Polyglot Persistence**<br>PostgreSQL for ACID-compliant relational data; MongoDB for high-volume, schema-flexible geolocation logs and delivery event streams. |
| **Admin Dashboard**<br>Full CRUD for restaurants, menus, and orders. Analytics view shows revenue charts, commission breakdown, and a Profit Tracker with CSV export. | **N+1 Query Fix**<br>Solved JPA/Hibernate N+1 via @EntityGraph annotations, collapsing O(n) database round-trips into single optimised JOIN fetch queries. |

## 04 INFRASTRUCTURE — DOCKER COMPOSE

9 containerised services powering the full local stack

The entire backend infrastructure is containerised via **Docker Compose**, enabling one-command local environment spin-up. The compose file orchestrates 9 services across databases, message broker, admin UIs, and distributed tracing.

| DATABASE SERVICES | | |
|---|---|---|
| **delivery-postgres**<br>■ postgres:16-alpine | Primary relational database for structured driver profiles, order records, and user data. | **5433:5432** |
| **delivery-mongodb**<br>■ mongo:7 | Document store for high-volume geolocation logs, delivery tracking events, and flexible schemas. | **27017** |

| ADMIN UIs | | |
|---|---|---|
| **delivery-pgadmin**<br>■ dpage/pgadmin4:latest | Web-based PostgreSQL admin panel. Login: admin@delivery.com | **5050:80** |
| **delivery-mongo-ui**<br>■ mongo-express | Mongo Express UI for MongoDB inspection. Login: admin / admin123 | **8081:8081** |

| MESSAGE BROKER |
|---|

| | | |
|---|---|---|
| **delivery-zookeeper**<br>■ confluentinc/cp-zookeeper:7.5.0 | Zookeeper for Kafka cluster coordination and leader election. | **2181** |
| **delivery-kafka**<br>■ confluentinc/cp-kafka:7.5.0 | Apache Kafka broker. Internal listener on 29092; external on 9092. Single-broker setup. | **9092** |
| **delivery-kafka-ui**<br>■ provectuslabs/kafka-ui | Kafka UI for topic inspection, consumer group monitoring, and message browsing. | **8090:80**<br>**80** |
| **OBSERVABILITY** | | |
| **delivery-zipkin**<br>■ openzipkin/zipkin | Distributed tracing UI. Collects spans for end-to-end request latency visibility. | **9411** |

**Kafka Connection Flow**

| CraveRush Core (Producer) | → Kafka Broker :9092 → | Delivery Service (Consumer) |
|---|---|---|
| Publishes: order.confirmed | Topic: order-events<br>3 partitions | Subscribes: order.confirmed |
| localhost:8080 | Zookeeper: :2181 | localhost:9090 |

## 05 SKILLS DEMONSTRATED

| | |
|---|---|
| **Frontend** | React · TypeScript · Tailwind CSS · Framer Motion · Zustand · React Query · Axios |
| **Backend** | Java · Spring Boot · Spring Security · JWT · Hibernate · @EntityGraph · REST APIs |
| **Messaging** | Apache Kafka · Event-Driven Architecture · Multi-Partition Consumers · Async Decoupling |
| **Databases** | PostgreSQL · MongoDB · Polyglot Persistence · Schema Design · ACID Transactions |
| **Payments** | Stripe API · PCI-Compliant Checkout · Webhook Handling |

| | |
|---|---|
| **DevOps** | Docker · Docker Compose · Containerised Dev Stack · Zipkin Tracing |
| **Architecture** | Microservices · Service Decoupling · Fault Isolation · Scalable Design |
| **Design** | Mobile-First UI · Dark Theme · Glassmorphism · Micro-Animations · Dribbble Aesthetic |

**This dual-application project demonstrates end-to-end ownership** — from pixel-perfect frontend design and secure stateless authentication, to event-driven microservice orchestration, polyglot persistence, and fully containerised infrastructure.